docker-lab — exemplo completo

Um projeto Python de exemplo, o `job.env` dele e os comandos para rodar, acompanhar, parar e ver o histórico.

Cada arquivo `.env` descreve um job, e um projeto pode ter **quantos arquivos `.env` precisar** (por exemplo `job.env`, `job_treino_grande.env`, `job_analise.env`), todos na raiz do projeto e cada um com um `JOB_NAME` diferente.

1. O projeto de exemplo

estrutura

```
~/meu_projeto/
├── job.env          ← fica na raiz
├── venv/            ← Python do projeto
├── dados/
│   └── iris.csv
├── resultados/
├── requirements.txt ← pacotes do projeto
└── treino.py        ← script a rodar
```

requirements.txt

```
pandas
scikit-learn
```

criar o venv (uma vez)

```
cd ~/meu_projeto
docker-lab venv 3.11 requirements.txt
```

Cria `venv/` com o Python escolhido (3.7 a 3.14) e instala os pacotes. Sem argumentos, `docker-lab venv` mostra um menu de versões. O venv só funciona pelo `docker-lab run`.

2. O job.env

Copie o modelo com `cp /opt/docker_lab/job.env.example ~/meu_projeto/job.env` e deixe assim:

~/meu_projeto/job.env

```
SCRIPT=treino.py              # script a rodar (relativo à pasta do projeto)
PYTHON_BIN=venv/bin/python    # Python do venv do projeto
CORES=2                       # quantos cores
MEMORY_GB=4                   # RAM máxima (passou disso, o job é encerrado)
GPU=none                      # "none" sem GPU, ou "0" para usar a GPU 0
#GPU_MEMORY_GB=8              # VRAM máxima: obrigatório quando GPU=0
JOB_NAME={USERNAME}_treino    # mantenha o {USERNAME}_, troque só o resto
NOTIFY_EMAIL=aluno@gmail.com      # obrigatório: avisos da fila por e-mail
ESTIMATED_RUNTIME_HOURS=1.5     # obrigatório: tempo estimado, em horas
```

3. Rodar (entra na fila)

Dentro da pasta do projeto. O experimento entra na **fila** e inicia sozinho quando houver recursos, como você, em segundo plano. Você pode fechar o terminal.

```
aluno@servidor:~/meu_projeto$ docker-lab run job.env
== Buildando imagem (docker-lab-runner:latest) ==
== Enfileirando 'aluno_treino' (cores=2 mem=4GB gpu=none) ==
== Experimento #14 (aluno_treino) entrou na fila ==
   Situação:    aguardando na fila — posição 2 de 3
   Estimativa:  informada 1h30m, corrigida 2h00m (sua margem atual: +30m)
   E-mails:     você receberá avisos em aluno@gmail.com (próximo da fila, início, fim)
   Ver o log:   docker-lab logs aluno_treino
Use "docker-lab queue" para acompanhar a fila
```

Se houver recursos livres, aparece `Situação: INICIOU IMEDIATAMENTE`. A **estimativa corrigida** é a sua estimativa mais a média dos seus erros anteriores (tempo real − estimado). Estimar bem ajuda seus jobs curtos a passarem na frente quando o primeiro da fila ainda não cabe.

4. A fila

```
aluno@servidor:~/meu_projeto$ docker-lab queue
EM EXECUÇÃO
    USUÁRIO                  EXPERIMENTO  CORES/RAM/VRAM        ESTIMADO  CORRIGIDO  RODANDO HÁ  RESTANTE
    -----------------------  -----------  --------------------  --------  ---------  ----------  ------------
    maria <maria@gmail.com>  maria_bert   8c / 32GB / GPU0 6GB  4h00m     4h30m      1h10m       3h20m
    jose (legado)            jose_antigo  4c / 16GB / -         -         -          2h05m       desconhecido

NA FILA
    POS  USUÁRIO                   EXPERIMENTO   CORES/RAM/VRAM  ESTIMADO  CORRIGIDO  ESPERANDO HÁ
    ---  ------------------------  ------------  --------------  --------  ---------  ------------
    1    ana <ana@gmail.com>       ana_grande    12c / 64GB / -  6h00m     6h00m      20m
 ➜  2    aluno <aluno@gmail.com>   aluno_treino  2c / 4GB / -    1h30m     2h00m      1m

RESUMO DA MÁQUINA
    RECURSO          TOTAL  TETO  EM USO  LIVRE
    ---------------  -----  ----  ------  -----
    Cores            16     12    12      0
    RAM (GB)         62     58    48      10
    VRAM GPU 0 (GB)  12     9     6       3

➜ = seus experimentos. Para cancelar ou parar: docker-lab stop
```

`➜` marca os seus. `legado` é um job iniciado antes da fila existir: ele ocupa recursos, mas o tempo restante é desconhecido. `excedido` no RESTANTE indica um job que já passou da estimativa corrigida.

5. Acompanhar o log

```
aluno@servidor:~/meu_projeto$ docker-lab logs job.env
Execuções de 'aluno_treino' (job.env):
    N  INICIO                FIM                   STATUS        SUGESTAO
    1  2026-10-08T10:02:11   2026-10-08T10:02:15   sem RAM       aumente MEMORY_GB (pediu 1GB)
    2  2026-10-08T10:30:05   -                     executando

== Últimas 50 linhas do log da execução 2 ==
n_estimators=50: acurácia=0.953
n_estimators=100: acurácia=0.960

== Log completo da execução 2: /opt/docker_lab/logs/aluno_treino_20261008_103005.log
   Ver tudo:             less "/opt/docker_lab/logs/aluno_treino_20261008_103005.log"
   Acompanhar ao vivo:   tail -f "/opt/docker_lab/logs/aluno_treino_20261008_103005.log"
   Ver outra execução:   docker-lab logs job.env <N>
```

Cada execução guarda o próprio log. `docker-lab logs job.env` lista todas as execuções desse `.env`, pergunta qual abrir (`Enter` abre a mais recente) e mostra as **últimas 50 linhas** dela. Para ver outra, passe o número: `docker-lab logs job.env 1`. Também aceita o nome do job, como aparece no `docker-lab history`: `docker-lab logs aluno_treino`.

6. Parar ou cancelar

```
aluno@servidor:~/meu_projeto$ docker-lab stop
Jobs em execução e na fila:
1) maria_bert (cores=[0, ..., 7] mem=32GB gpu=0 vram=6GB) — EM EXECUÇÃO
2) aluno_treino — NA FILA (posição 2, de aluno, cores=2 mem=4GB)
3) Sair
Qual você quer parar/cancelar (número, ou 'Sair')? 2
Cancelado: o experimento saiu da fila (se submeter de novo, entra no fim).
```

Parar um job em execução **encerra** o job: não é possível retomá-lo depois. Cancelar um job da fila faz ele **perder o lugar**.

7. Histórico

```
aluno@servidor:~/meu_projeto$ docker-lab history
JOB_NAME          USUARIO  INICIO               FIM                  STATUS        SUGESTAO
-------------------------------------------------------------------------------------------------------------
aluno_treino      aluno    2026-10-08T10:02:11  2026-10-08T10:02:15  sem RAM       aumente MEMORY_GB (pediu 1GB)
aluno_treino      aluno    2026-10-08T10:30:05  2026-10-08T10:52:19  interrompido
aluno_analise     aluno    2026-10-08T11:04:40  2026-10-08T11:31:02  executou
```

`executou`	terminou sem erro
`erro`	o script falhou: veja o motivo com `docker-lab logs`
`sem RAM`	passou do `MEMORY_GB`: aumente no `.env`
`sem VRAM`	passou do `GPU_MEMORY_GB` (`CUDA out of memory`): aumente no `.env` ou reduza o batch size
`interrompido`	foi parado com `docker-lab stop`, ou a máquina reiniciou